UNIVERSITÀ di VERONA

PHD School in NATURAL SCIENCES AND ENGINEERING

Brain Computer Interfaces

Laboratory

Dr. Lorenza Brusini

Dept. of Computer Science, University of Verona
email: lorenza.brusini@univr.it

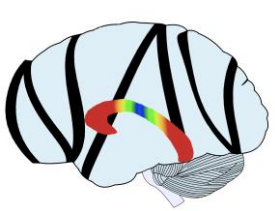
You can find me at @

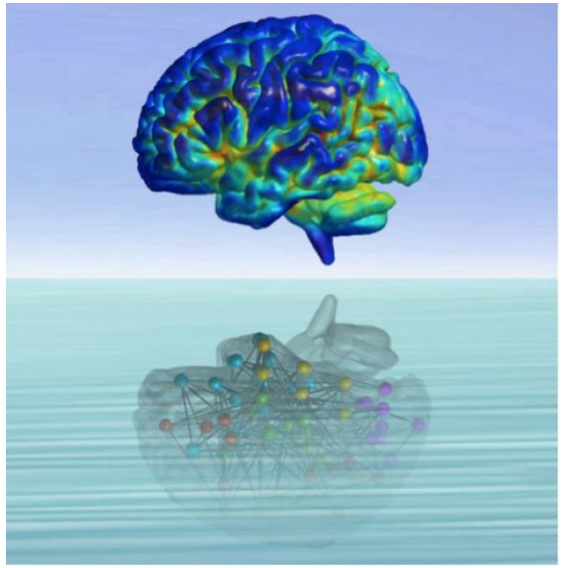
First floor Ca’ Vignal 2, room 1.64B

Neuroimaging Lab (CV2 piano -1)

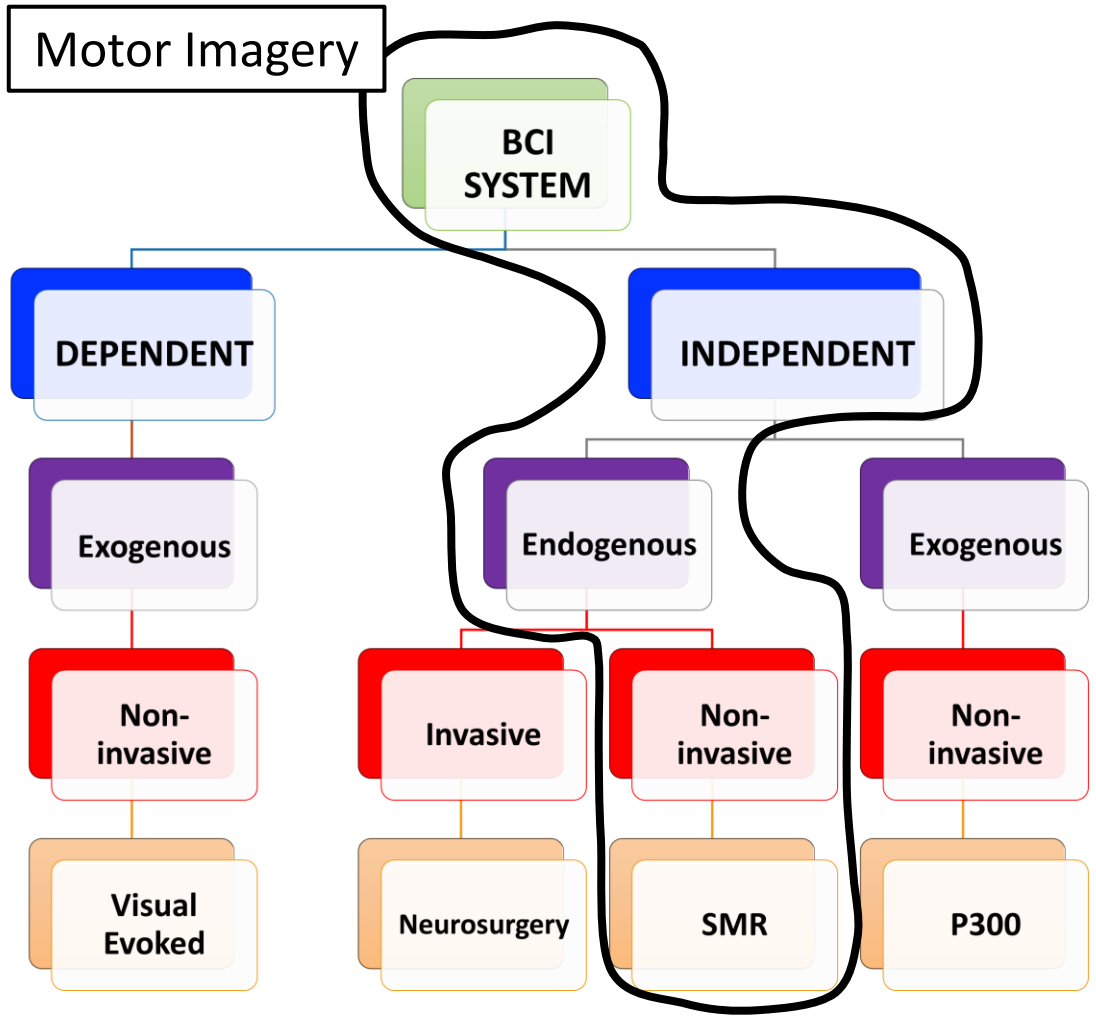
Vision, Image Processing & Sound Lab, VIPS (CV2 piano -2)

http://vips.sci.univr.it





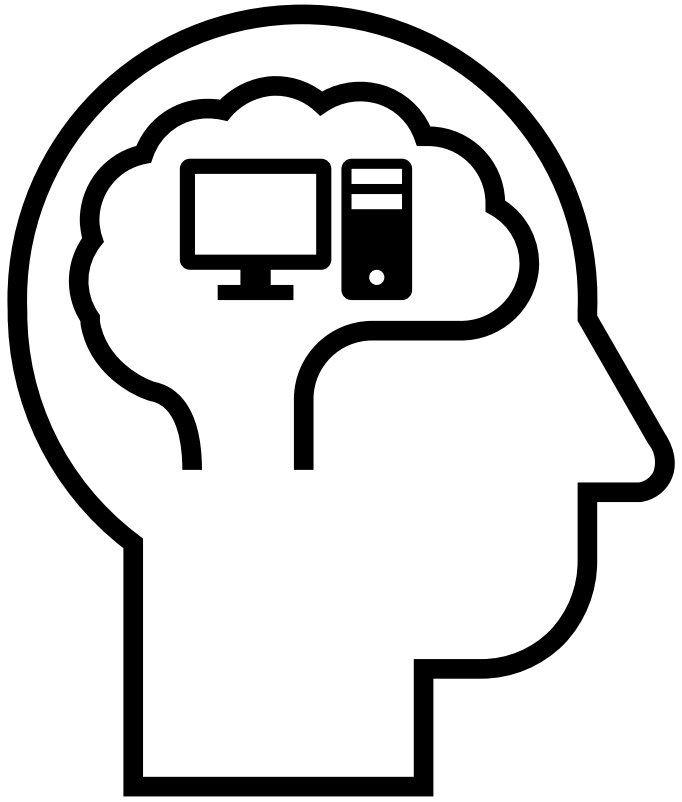
Approach of the laboratory



Approach of the laboratory

The lectures target at:

- Learning from code
- Giving some necessary theoretical details
- Practicing on the code using real data
    - Data recorded in the context of the BCI Competition will be used along the course (publicly avaliable at: http://www.bbci.de/competition/)

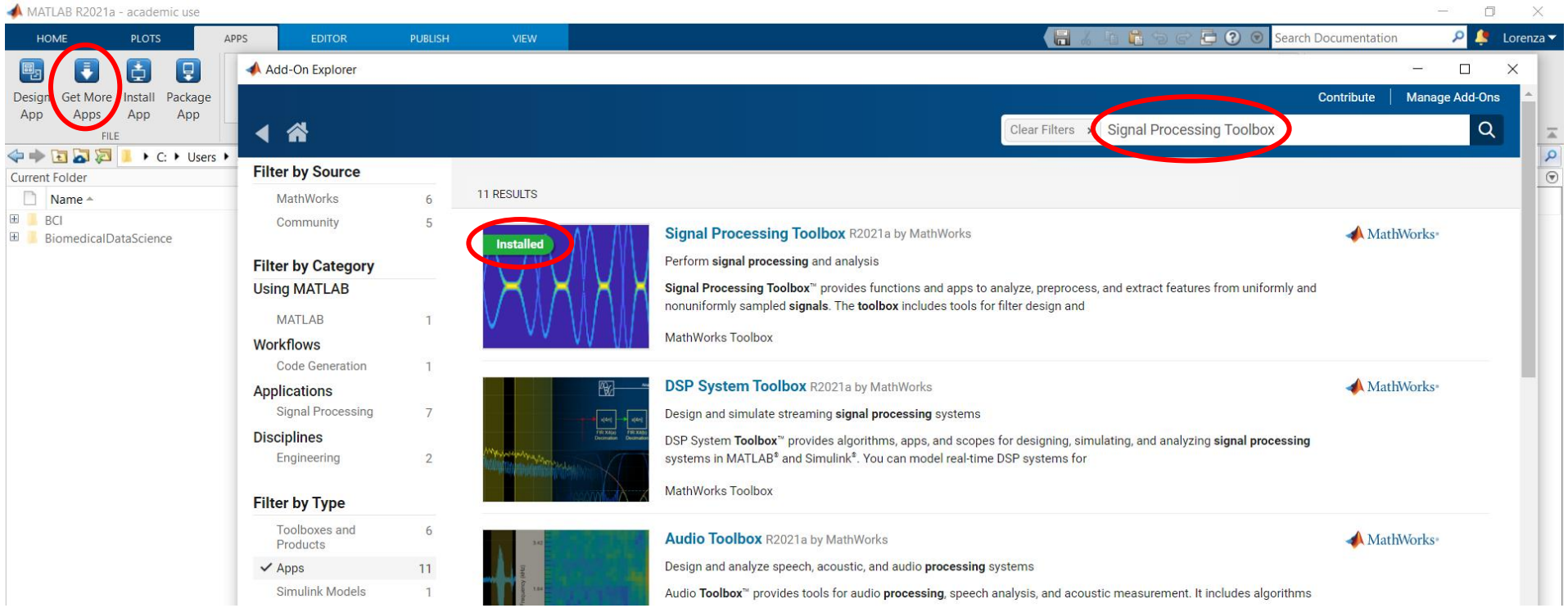


Final evaluation

- On the 9th of September, part of the lecture will be dedicated to the final evaluation
- The final evaluation will consist in the analysis of a new data set by using the notions learned along the course
- The new data set for the final evaluation will be always provided by the public database stored in the context of the BCI Competition.
- However,
    - Fantasy is welcome!
    - If desired, own data set can be used!

Tools

- The course will be based on MATLAB software and two of the MATLAB add-on toolboxes:
  - Signal Processing Toolbox
  - Statistics and Machine Learning Toolbox



Tools

- Moreover, the MATLAB toolbox EEGLAB will be used

Download and installation:

1. Provided at: https://sccn.ucsd.edu/eeglab/downloadtoolbox.php
2. Unzip the EEGLAB zip file in the folder of your choice
3. Start Matlab
4. Add the EEGLAB folder just uncompressed to the MATLAB path

```
addpath('eeglab2021.1')
```

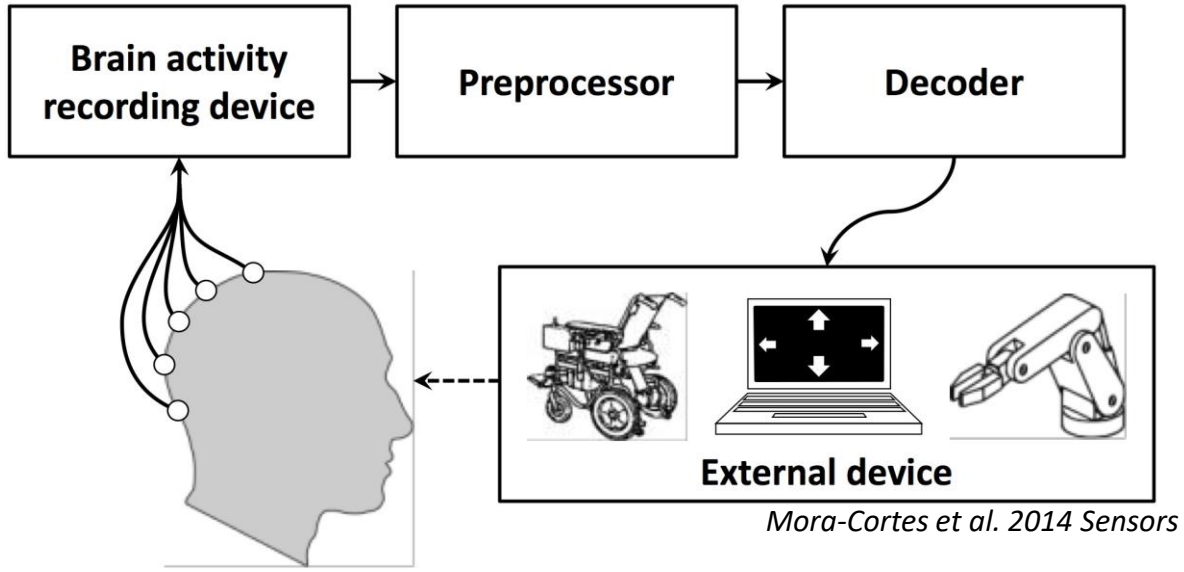
Let’s start…

For any question, just ask or type in the ZOOM chat!

The BCI model

A BCI comprises:

- A **device** that records the brain activity (invasive/non-invasive)
- A **preprocessor** that reduces noise and artifacts, prepares the signals for further processing and extracts the relevant information from the recordings

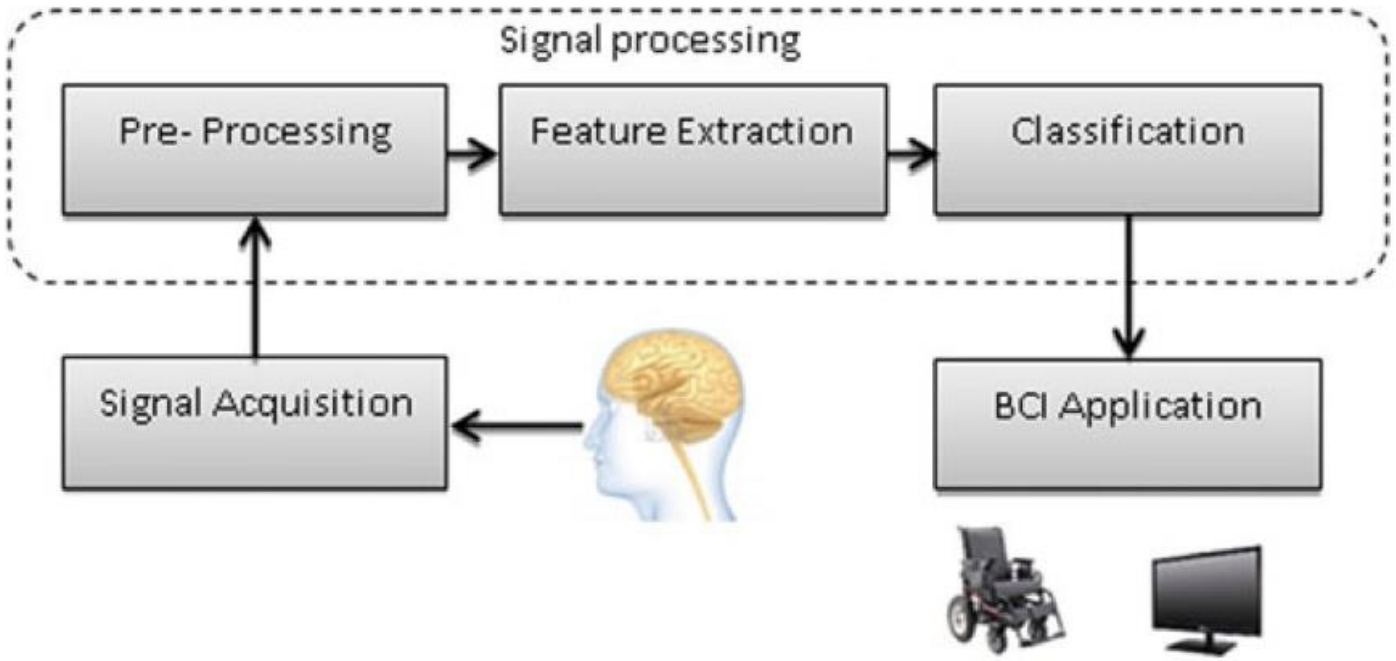


Mora-Cortes et al. 2014 Sensors

- A **decoder** that classifies the extracted relevant information into a control signal for an **external device** that could be any type BCI-compatible application (e.g., a robotic actuator, a prosthesis, a computer screen etc.), and that provides feedback to the user

Design and implementation of a BCI

- **Preprocessing**
  (artifact removal, filtering…)
- **Feature extraction in time or frequency domain**
  (Frequency band power, autoregressive parameters, wavelet…)
- **Classification**
  (threshold, support vector machine, neural networks)



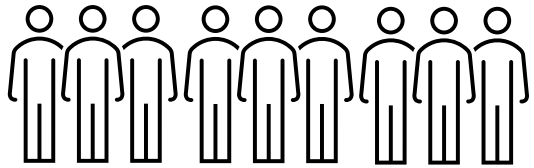
Preprocessing

- Data set description
- EEGLAB
- Filtering
- Re-reference
- Epochs extraction

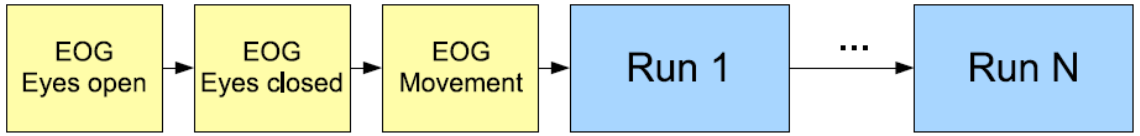
Data set description: experimental paradigm

BCI Competition IV 2008, data set 2a

Subjects



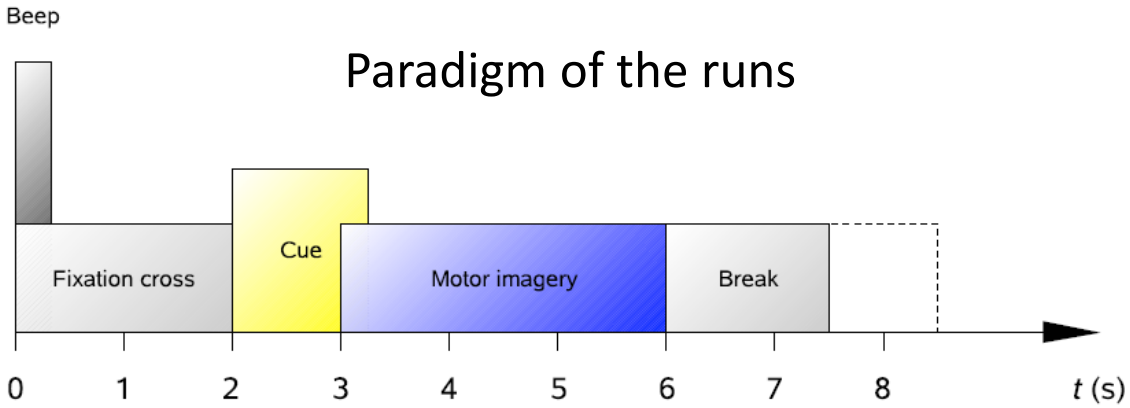
Recording of two sessions performed as follows:



BCI Competition documentation

Motor imagery tasks

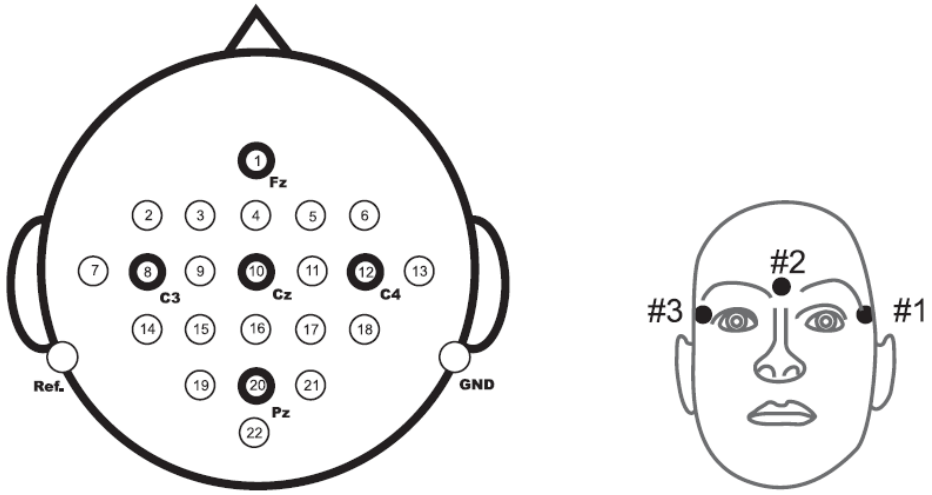
Paradigm of the runs



BCI Competition documentation

Data set description: data recording

- Signals **sampling**: 250 Hz + bandpass-filtering: 0.5 Hz - 100 Hz
- Notch filter to suppress line noise: 50 Hz
- Amplifier sensitivity: 100 µV
- **EOG channels** for subsequent artifact processing
- Recorded data stored in **.gdf** (General Data Format) files



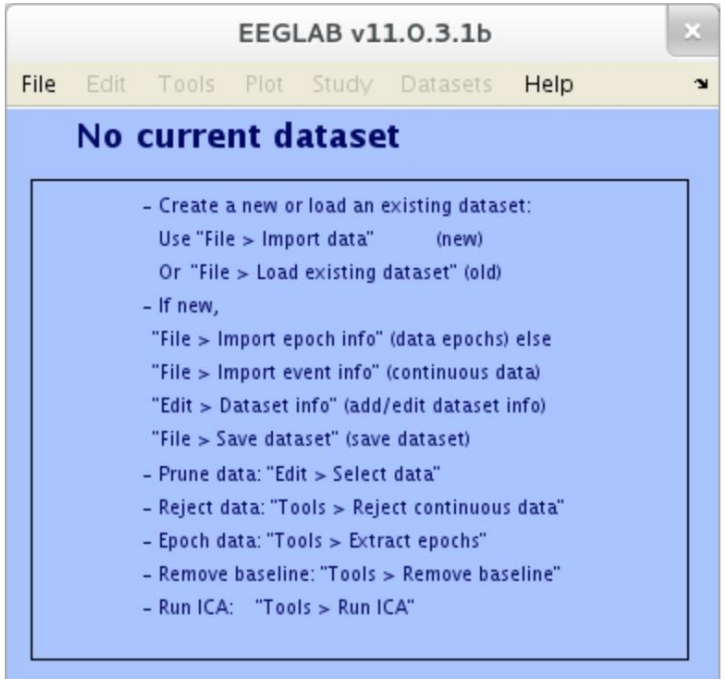
BCI Competition documentation

EEGLAB: initialization

```
% EEGLAB initialization
[ALLEEG, EEG, CURRENTSET, ALLCOM] = eeglab;
```

- **ALLEEG:** array containing all the loaded EEG data sets
- **EEG:** current EEG data set
- **CURRENTSET:** index of the current data set
- **ALLCOM:** all the commands issued from the EEGLAB menu

Alternatively, the graphic user interface can be used by typing "**eeglab**" on the MATLAB command window and pressing enter.



EEGLAB: data set

```
% Loading the eeg data from file in a dedicated structure
EEG = pop_biosig(filename);
```

EEG is a structure with some important fields like:

- **nbchan:** number of channels
- **srate:** sampling rate (250 Hz in this case)
- **times:** time at each sampled point (4 ms in this case, giving dimensions of the variable equal to 1x672528)
- **data:** EEG signal value for each time point (in this case, the dimensions of the variable are 25x672528)
- **event:** structure containing information about the recorded events
- **chanlocs:** structure containing information about the recording system

Some fields of the EEG structure must be filled with external information like, e.g., chanlocs (see the next slides).

EEGLAB: data set

Number of identification depending on its description

Starting position in the data vector (where each position represents a sample)

EEG.event

Fields	type	edftype	edfplustype	latency	duration	urevent
1	'eeg:Idling EEG - eyes open'	276	[]	1	29682	1
2	'boundary'	32766	'start of a new segment (after a break)'	29684	0	2
3	'eeg:Idling EEG - eyes closed'	277	[]	29684	20271	3
4	'boundary'	32766	'start of a new segment (after a break)'	49956	0	4
5	'1072'	[]	[]	49956	41562	5
6	'boundary'	32766	'start of a new segment (after a break)'	91519	0	6
7	'Start of Trial, Trigger at t=0s'	768	[]	91869	1875	7
8	'class4, Tongue- cue onset (BCI experiment)'	772	[]	92369	313	8
9	'Start of Trial, Trigger at t=0s'	768	[]	93872	1875	9
10	'class3, Foot, towards Right - cue onset (BCI experiment)'	771	[]	94372	313	10
11	'Start of Trial, Trigger at t=0s'	768	[]	95790	1875	11
12	'class2, Right hand- cue onset (BCI experiment)'	770	[]	96290	313	12

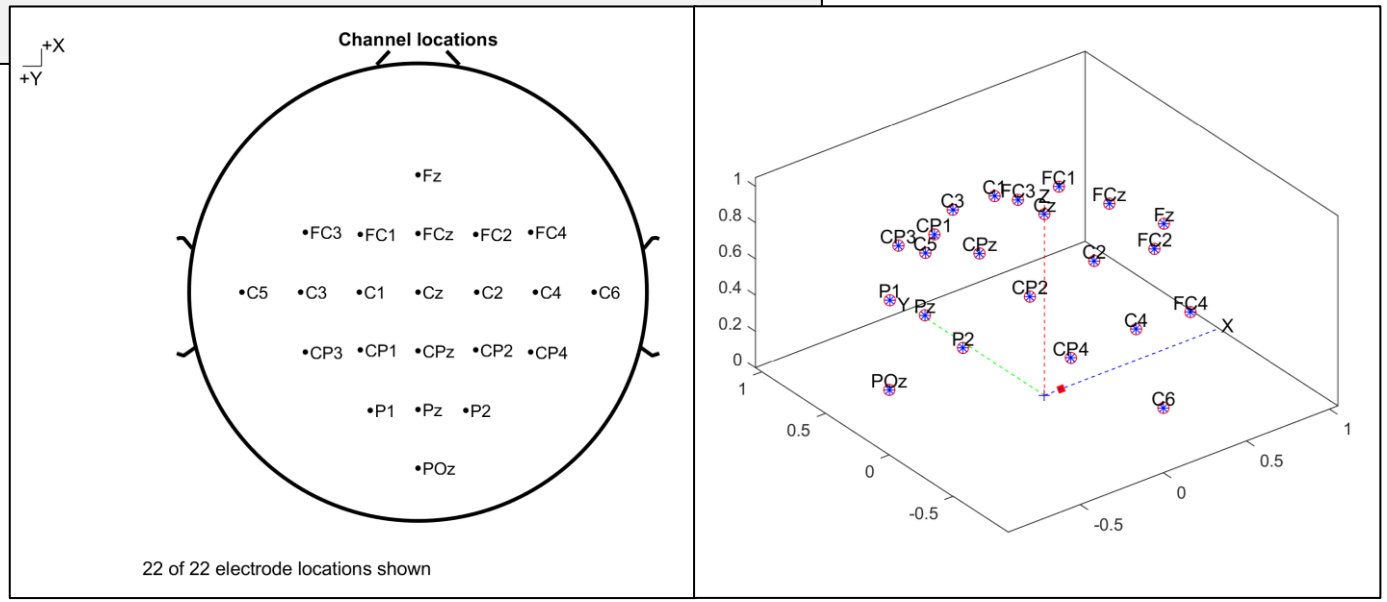
Description

Number of samples along which the event in object has been recorded

EEGLAB: data set

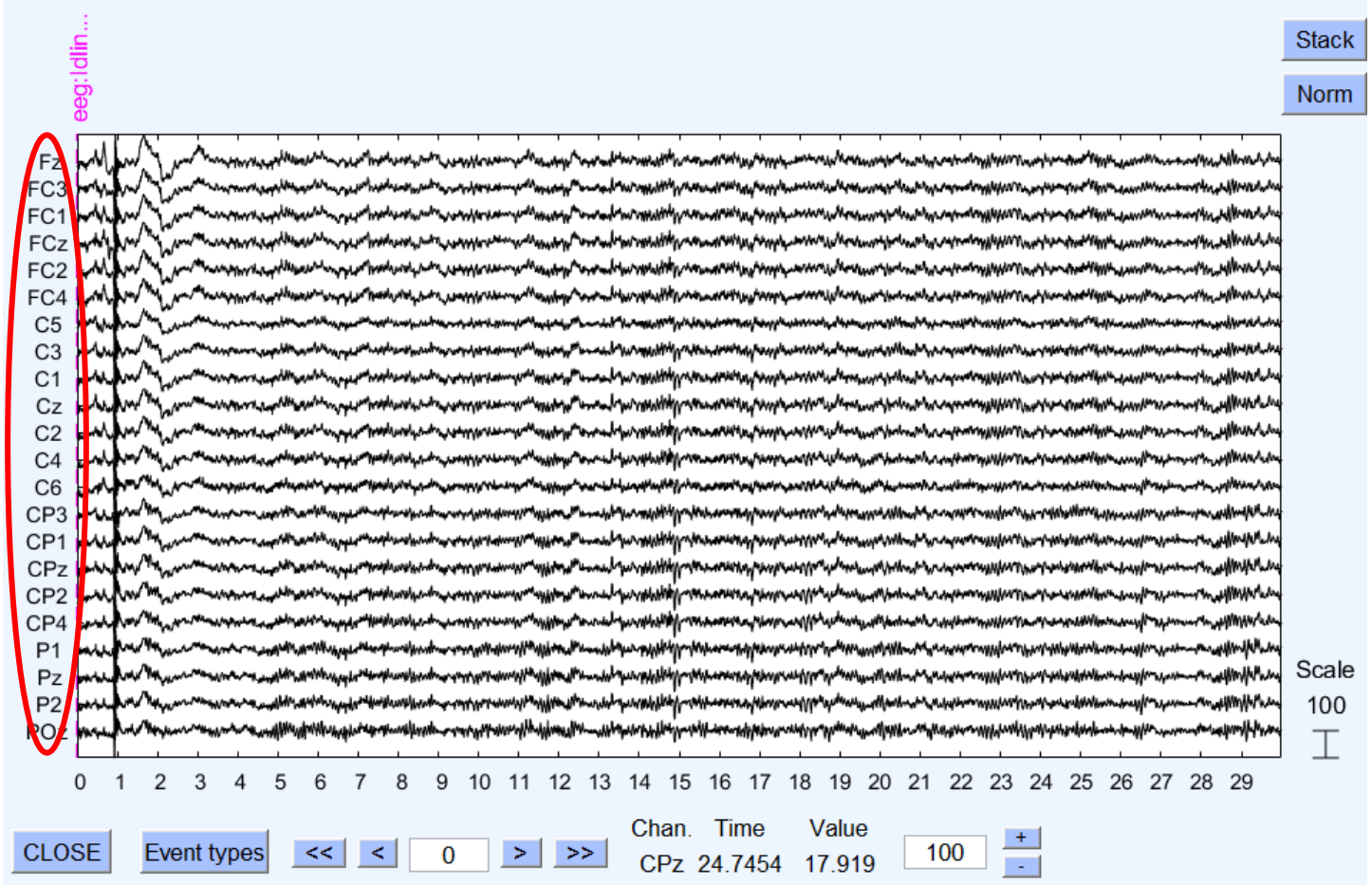
```
% Discarding the channels corresponding to the three monopolar EOG
% channels
EEG = pop_select(EEG, 'nochannel', [23, 24, 25]);

% Update the field dedicated to channels using the file .locs
chanlocs22 = readlocs('chanlocs22.locs');
EEG.chanlocs = chanlocs22;
```

Defined by a **label** and a **position** (in both Cartesian and spherical coordinates) for each channel



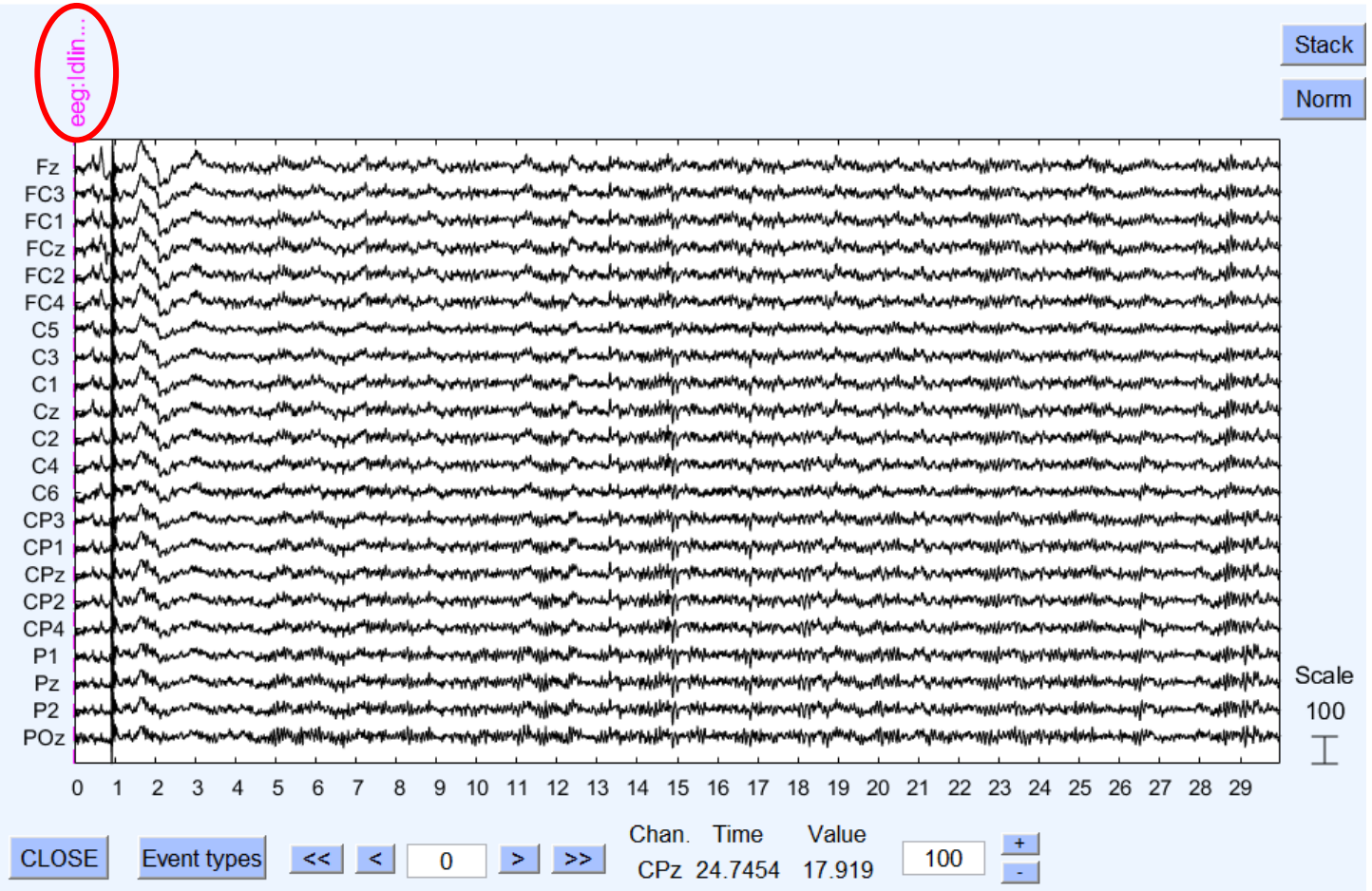
EEGLAB: data set

```
% Displaying the raw eeg signal
eegplot(EEG.data,...
    'title', 'Raw EEG',...
    'srate', EEG.srate,...
    'eloc_file', EEG.chanlocs,...
    'events', EEG.event,...
    'winlength', winlength,...
    'color', color,...
    'spacing', 100);
```



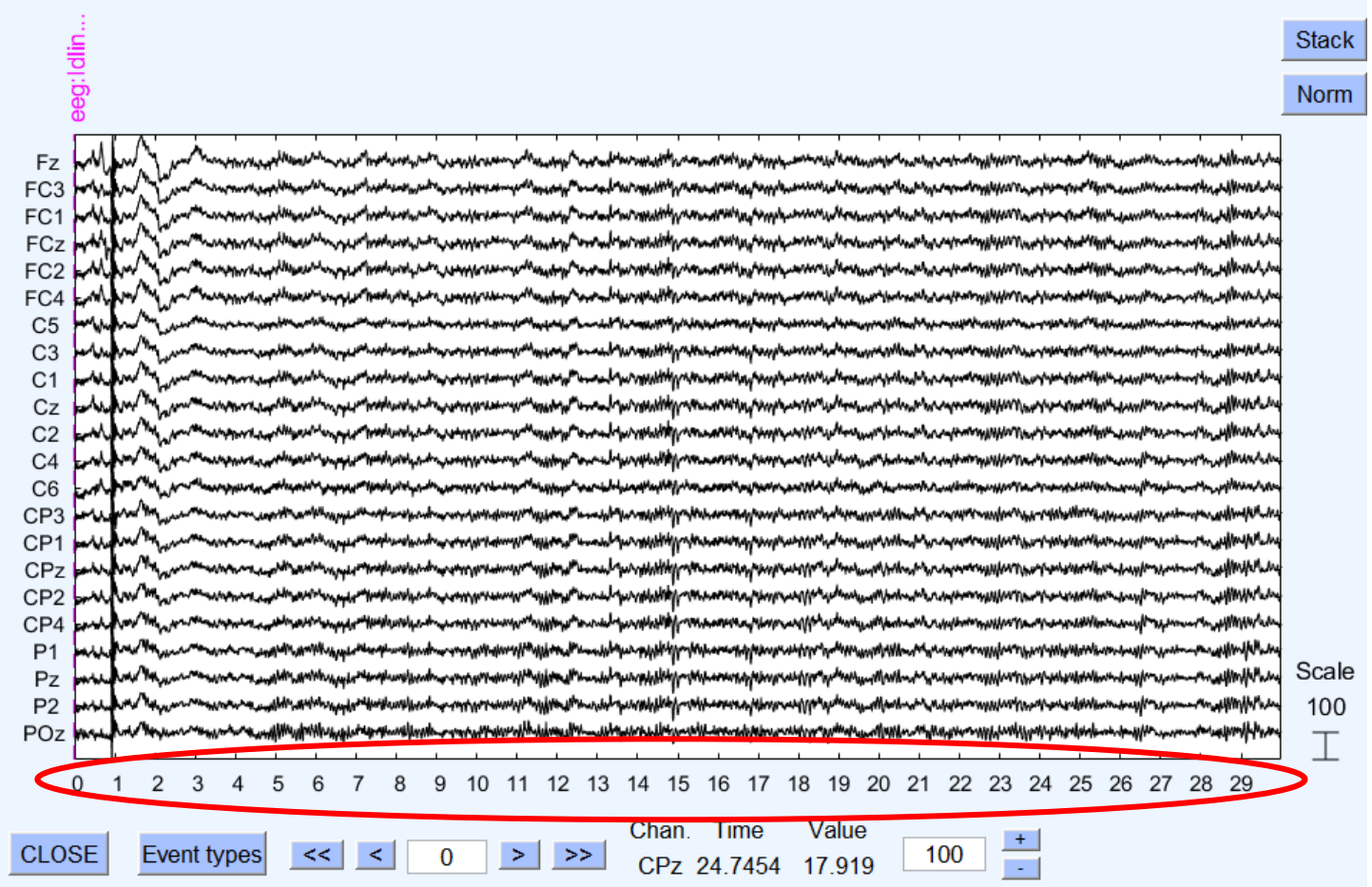
EEGLAB: data set

```
% Displaying the raw eeg signal
eegplot(EEG.data,...
     'title', 'Raw EEG',...
     'srate', EEG.srate,...
     'eloc_file', EEG.chanlocs,...
     'events', EEG.event,...
     'winlength', winlength,...
     'color', color,...
     'spacing', 100);
```



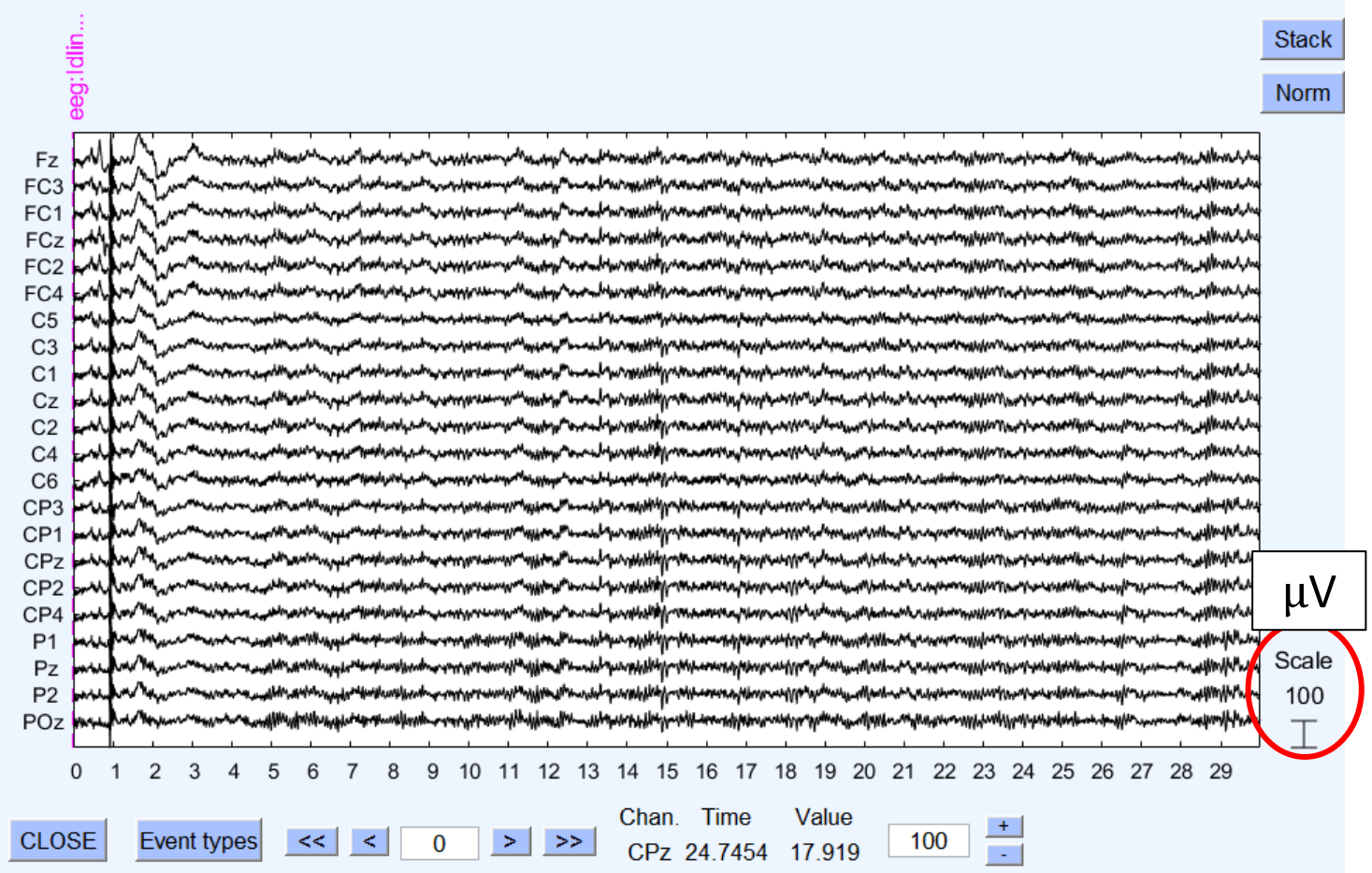
EEGLAB: data set

```
% Displaying the raw eeg signal
eegplot(EEG.data,...
     'title', 'Raw EEG',...
     'srate', EEG.srate,...
     'eloc_file', EEG.chanlocs,...
     'events', EEG.event,...
     'winlength', winlength,...
     'color', color,...
     'spacing', 100);
```



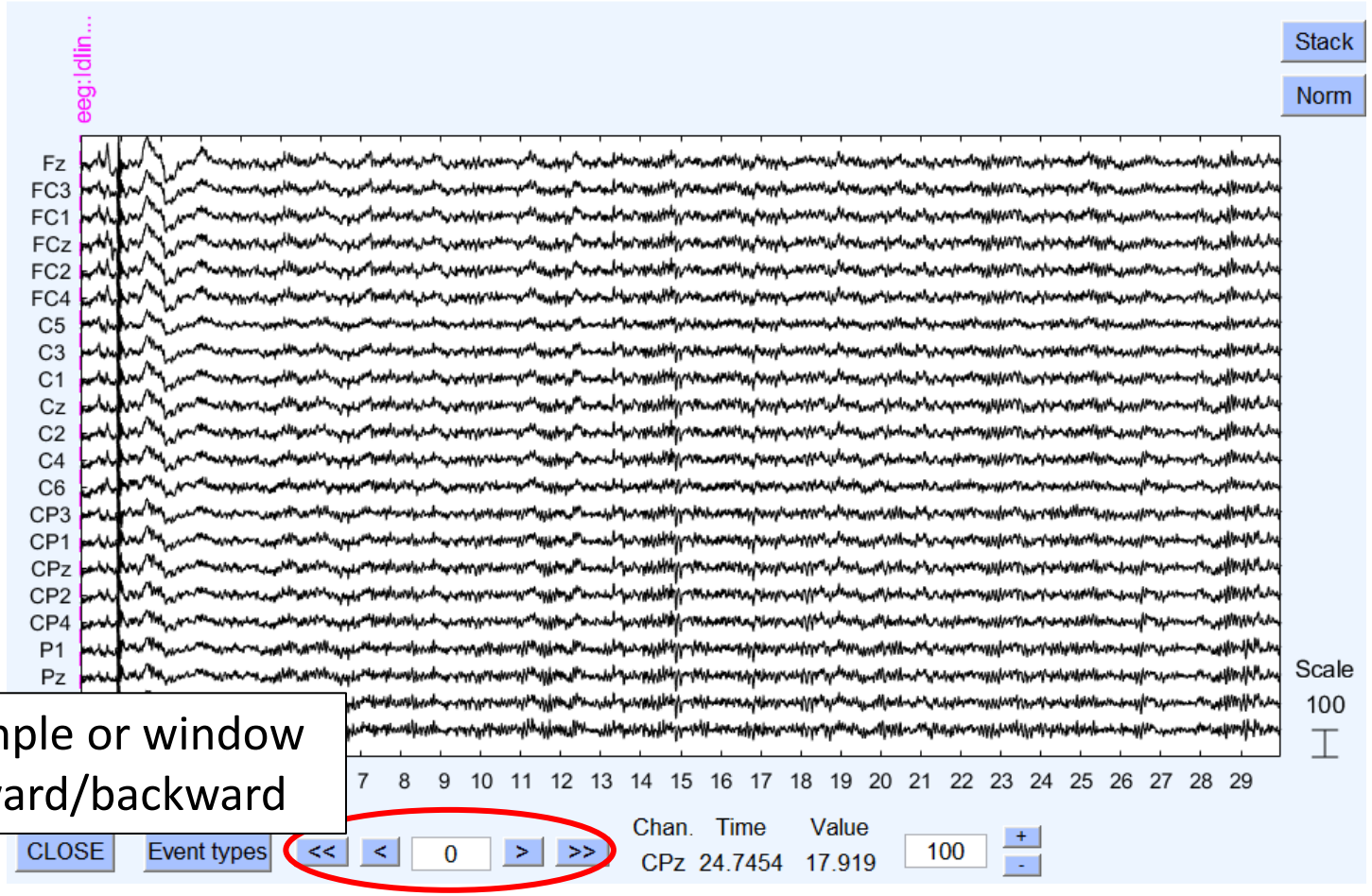
EEGLAB: data set

```
% Displaying the raw eeg signal
eegplot(EEG.data,...
     'title', 'Raw EEG',...
     'srate', EEG.srate,...
     'eloc_file', EEG.chanlocs,...
     'events', EEG.event,...
     'winlength', winlength,...
     'color', color,...
     'spacing', 100);
```



EEGLAB: data set

```
% Displaying the raw eeg signal
eegplot(EEG.data,...
    'title', 'Raw EEG',...
    'srate', EEG.srate,...
    'eloc_file', EEG.chanlocs,...
    'events', EEG.event,...
    'winlength', winlength,...
    'color', color,...
    'spacing', 100);
```



1 sample or window forward/backward

EEGLAB: useful commands

```
% Storing the current EEG data
[ALLEEG, EEG, CURRENTSET] = eeg_store(ALLEEG, EEG);
```

- **ALLEEG:** now contains, in addition, the data set stored in EEG (the current one)
- **CURRENTSET:** index to the current set

```
% Creating a new EEG copy
[ALLEEG, EEG, CURRENTSET] = pop_newset(ALLEEG, EEG, CURRENTSET, 'setname',
'EEG_BAS');
```

- **EEG:** the EEG structure manipulated until now is renamed as ‘EEG_BAS’
- **ALLEEG:** now contains, in addition, the data set ‘EEG_BAS’
- **CURRENTSET:** the current set, at this point, could be the second one in ALLEEG

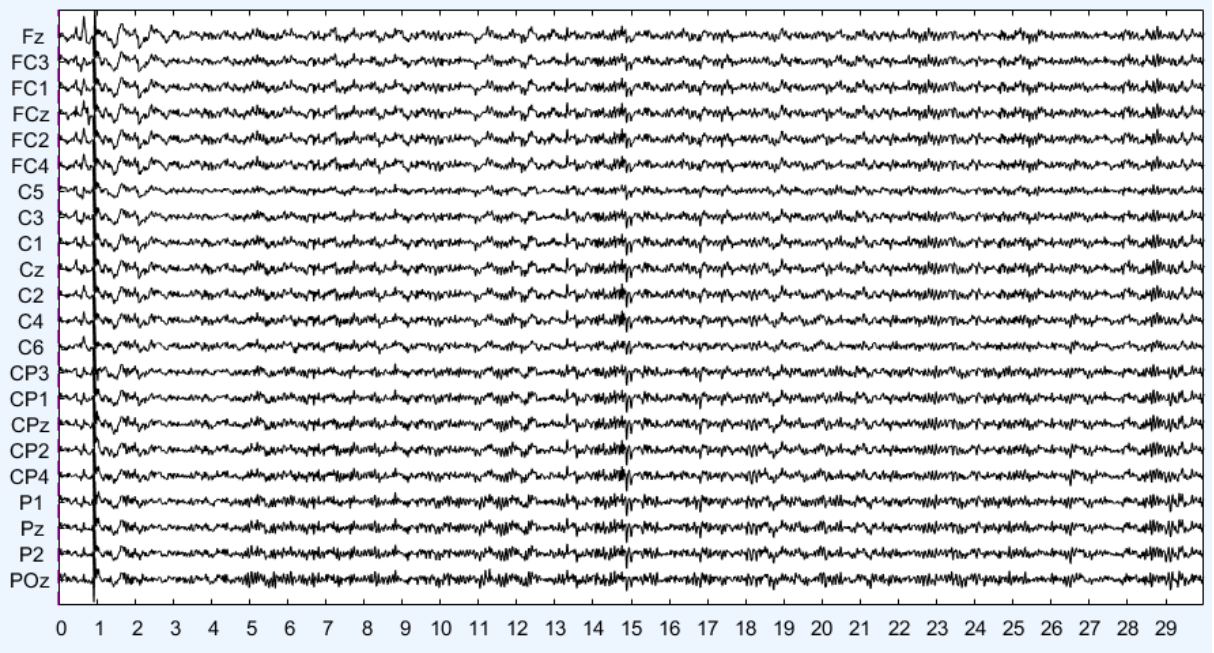
Filtering

1. Baseline removal
   - Do the mean of the signal for each channel
   - For each signal value, subtract the corresponding mean for all the channels

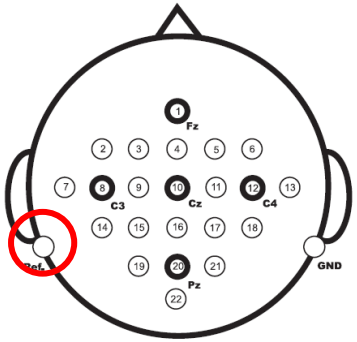
2. Retain frequencies in band 1-30 Hz

   In EEGLAB is adviced to do:

```
% High pass filter
EEG = pop_eegfilt(EEG, 1, 0);
% Low pass filter
EEG = pop_eegfilt(EEG, 0, 30);
```

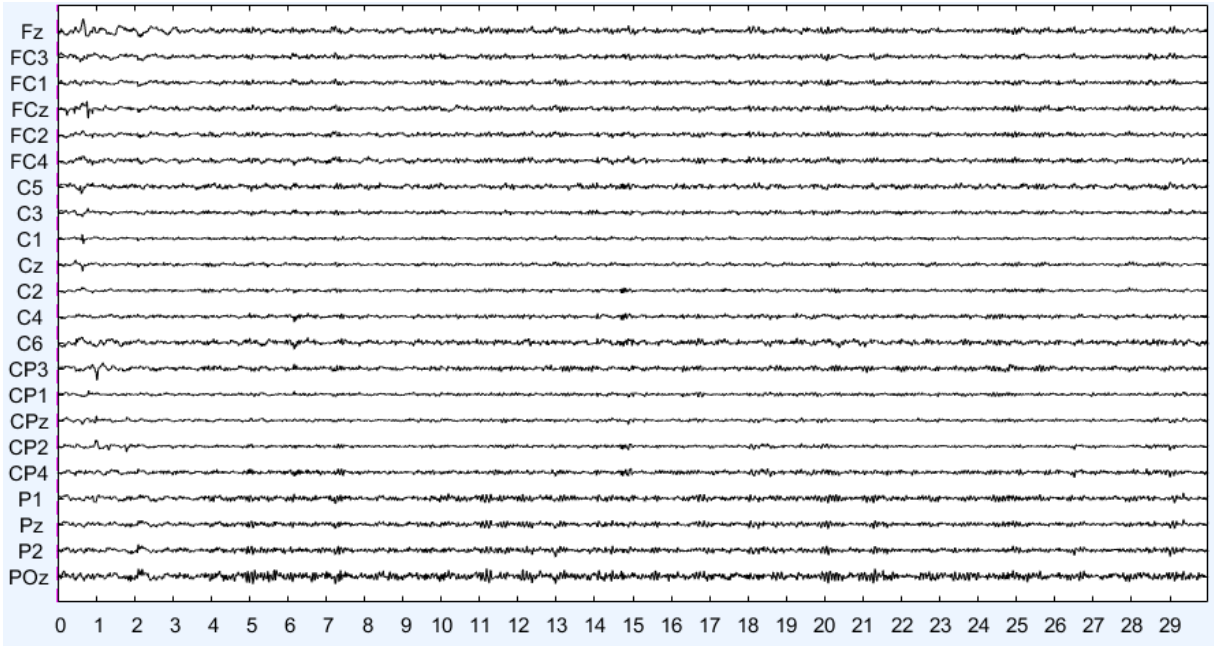


Re-reference



BCI Competition documentation

Classical EEG signal data of each channel are recorded as a difference between the electrical potential in the channel location and a **reference**.

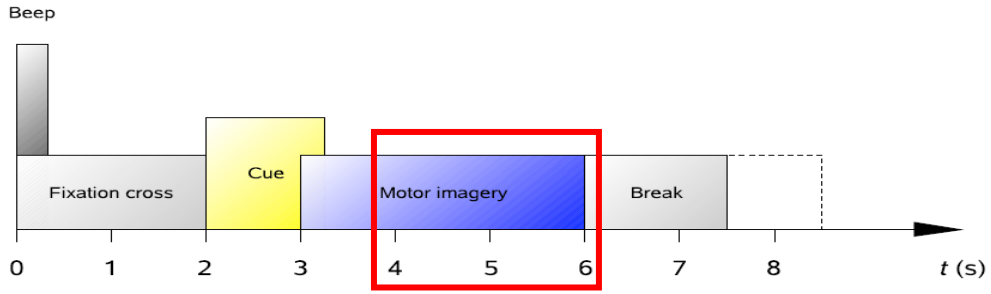



```
%   Re-referencing   the   data   with
respect to average reference
EEG = pop_reref(EEG, []);
```

Epochs extraction

The interesting events are those corresponding to motor imagery tasks:

- Left Hand: `EEG.event(i).edftype = 769`
- Right Hand: `EEG.event(i).edftype = 770`
- Both Feet: `EEG.event(i).edftype = 771`
- Tongue: `EEG.event(i).edftype = 772`



BCI Competition documentation

```
switch EEG.event(i).edftype
 case 769
  EEG.event(i).type = "Left_Hand";
  EEG.event(i).latency = EEG.event(i).latency + sampling*2;
  EEG.event(i).duration = sampling*2;
[...]
% EEG.data(:, EEG.event(i).latency:(EEG.event(i).latency + EEG.event(i).dur
% ation - 1)));
```

Epochs extraction

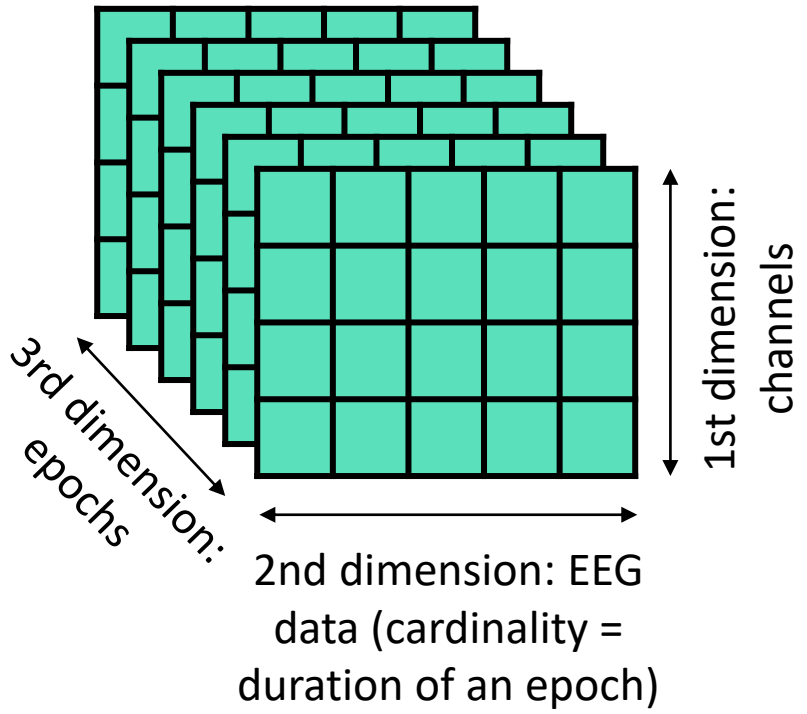
Let’s save the events of interest in a EEG field called epoch…

And focus, e.g., on the event “Right Hand”

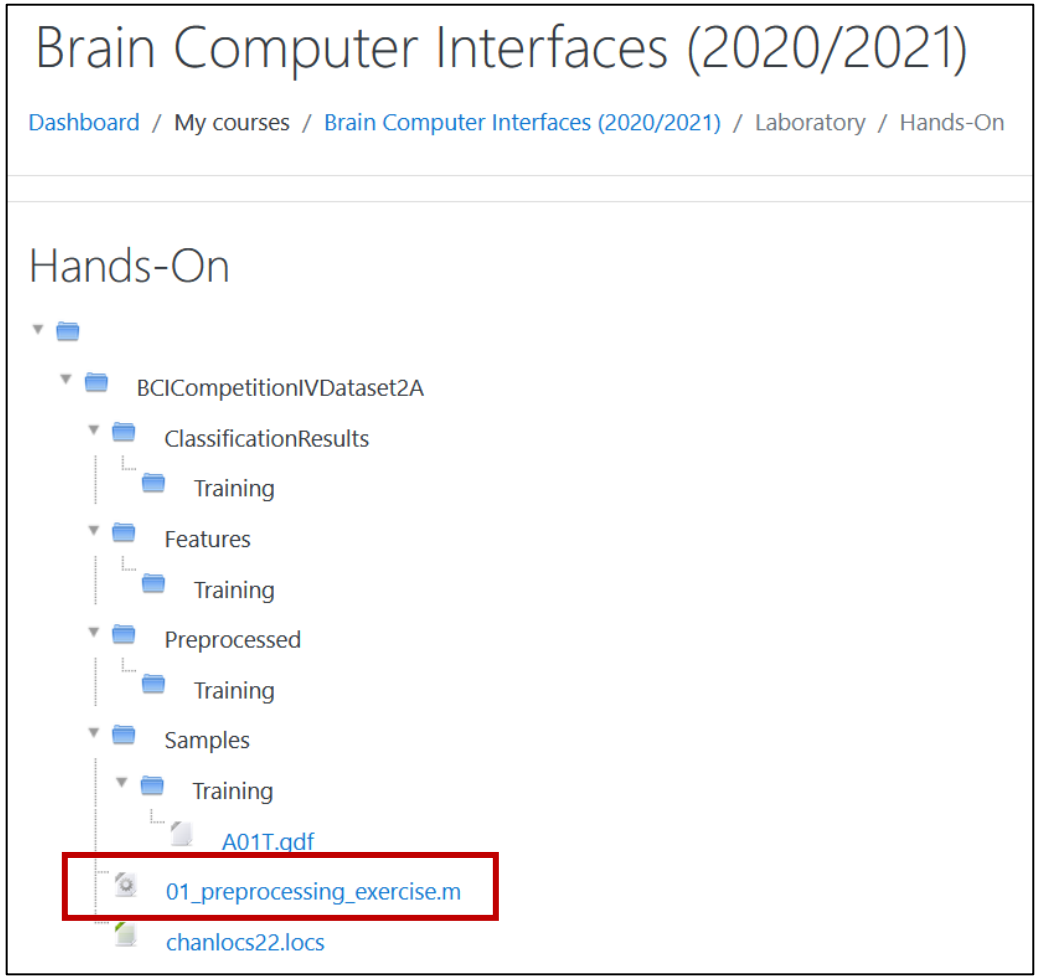
```
% Access to epochs belonging to the
% event of interest
EEG.epoch(3).data(:,:,1);
```

EEG.epoch

Fields	type	data
1	"Tongue"	22x500x72 single
2	"Both_Feet"	22x500x72 single
3	"Right_Han...	22x500x72 single
4	"Left_Hand"	22x500x72 single



Exercise

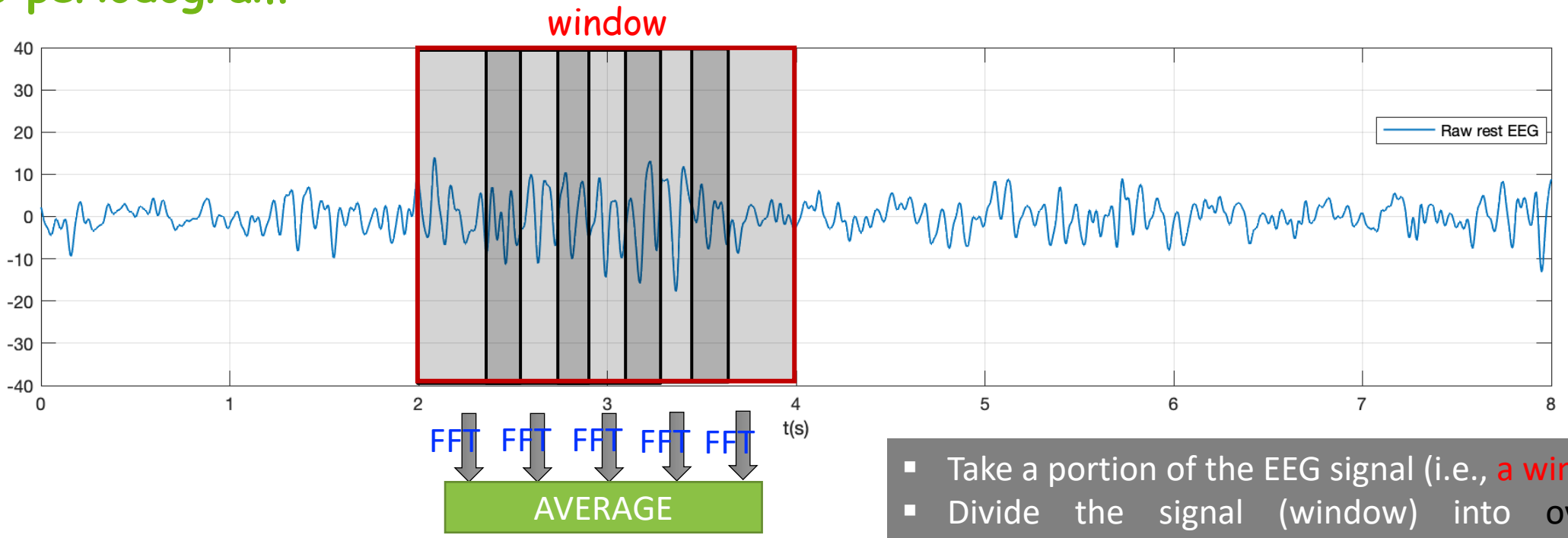

Brain Computer Interfaces (2020/2021)
Dashboard / My courses / Brain Computer Interfaces (2020/2021) / Laboratory / Hands-On
Hands-On
BCICompetitionIVDataset2A
ClassificationResults
Training
Features
Training
Preprocessed
Training
Samples
Training
A01T.gdf
01_preprocessing_exercise.m
chanlocs22.locs

Feature extraction

- Power Spectral Density
- Coherence
- Correlation
- Feature storing

Power Spectral Density

Welch's periodogram



PSD [channels x freq]

- Take a portion of the EEG signal (i.e., a window)
- Divide the signal (window) into overlapping segments
- Window each segment and compute the FFT of the signal for each segment
- Average the periodograms of the segment

Power Spectral Density

EEGLAB provides the function **spectopo**:

```
[spectra,freqs] = spectopo(data, frames, srate, 'overlap');
```

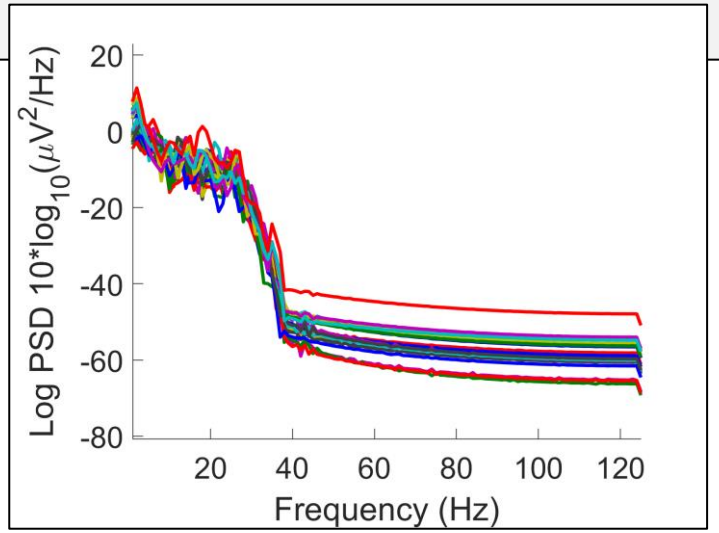
Input

- **data:** it can be a 2-D matrix corresponding to a single epoch
- **frames:** number of samples in an epoch
- **srate:** sampling rate
- **'overlap':** desired overlapped samples

Output

- **spectra:** power spectra in dB (nchans x nfreqs)
- **freqs:** frequencies of the spectra in Hz

```
% Calculate the power spectral density for
% all the epochs
for i = 1:nepochs
 [spectra_RH(:,:,i),freqs_RH(:,:,i)] =
       spectopo(EEG.epoch(3).data(:,:,i),
       length(EEG.epoch(3).data(:,:,i)),
       EEG.srate, 'overlap', EEG.srate/2);
end
```



Power Spectral Density

The EEGLAB function spectopo, internally calls the MATLAB function **pwelch**:

```
[spectra,freqs]   =   pwelch(data,
window, overlap, nfft, srate);
```

Input

- **window:** if it is an integer, pwelch is divided into segments of length equal to the integer value
- **nfft:** number of DFT points

```
for i = 1:nepochs
 [spectra_tmp,freqs_RH(:,:,i)] =
       pwelch(EEG.epoch(3).data(:,:,i)',
       EEG.srate, EEG.srate/2, EEG.srate,
       EEG.srate);
 spectra_RH(:,:,i) = 10*log10(spectra_tmp');
end
```

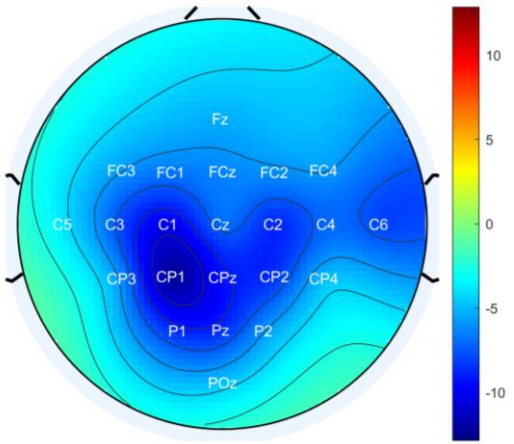
Power Spectral Density

It is usual to calculate the **mean** PSD over specific **band** frequencies:

```
% Retrieve indices for alfa frequencies
idx_alfa = (freqs_RH(:,1,1) >= 8 & freqs_RH(:,1,1) < 13);
% Calculate the mean PSD over alfa band
psd_RH_alfa = mean(spectra_RH(:,idx_alfa,:),2);
```

- Now, psd_RH_alfa has dimensions: nchans x 1 x nepochs

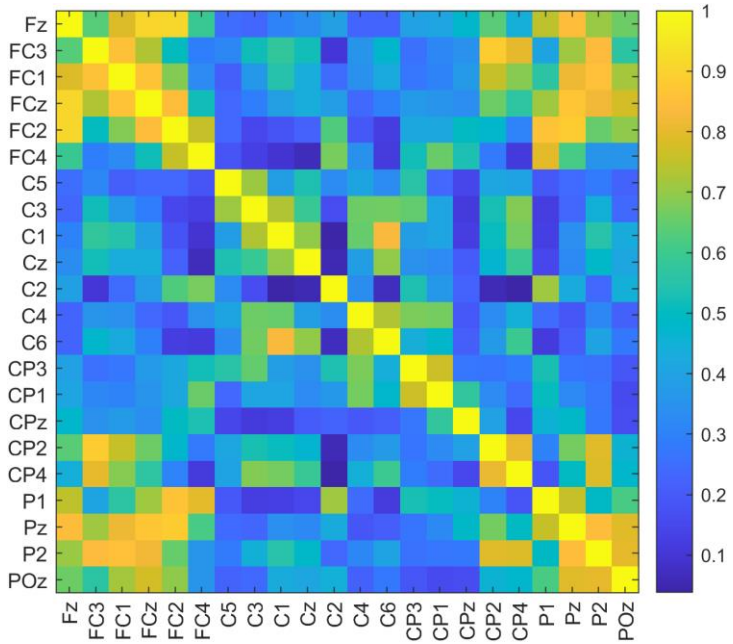
```
% Visualize a topographic scalp map of the "Right Hand"
% PSD alfa in the first epoch
topoplot(psd_RH_alfa(:,1,1), 'chanlocs22.locs',
      'electrodes', 'labels', 'ecolor', 'white');
```



Coherence

COHERENCE (or MAGNITUDE-SQUARE COHERENCE)

is a measure of degree of association or coupling of frequency spectra between different times series. The **coherence** is intended as the ratio between the cross-spectrum modulus and the product of the two autospectra (the coherence is basically a normalized cross spectrum).



Coherence

MATLAB provides the function mscohere

```
[coh, freqs] = mscohere(x, data, window, overlap, nfft, srate);
```

- **x:** vector containing the signal coming from one channel
- **coh:** magnitude-squared coherence estimate; according to the presented input, it has dimensions freqs x nchans

```
coh_alfa_RH = ones([EEG.nbchan,EEG.nbchan,nepochs]);
for i = 1:nepochs
 for j = 1:EEG.nbchan
  channel = EEG.epoch(3).data(j,:,i)';
  data = EEG.epoch(3).data(:,:,i)';
  [coh,f] = mscohere(channel, data, EEG.srate, EEG.srate/2, EEG.srate, EEG.srate);
  coh_alfa_RH(j,:,i) = mean(coh(idx_alfa, :), 1);
 end
end
```

Correlation

CORRELATION

Let s_x and s_y denote, respectively, the standard deviations and cov_{xy} the covariance of two time series x and y, the functional connectivity simply corresponds to the **Pearson's correlation coefficient r** of x and y

$$r_{xy} = \frac{cov_{xy}}{s_x s_y} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2 \sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

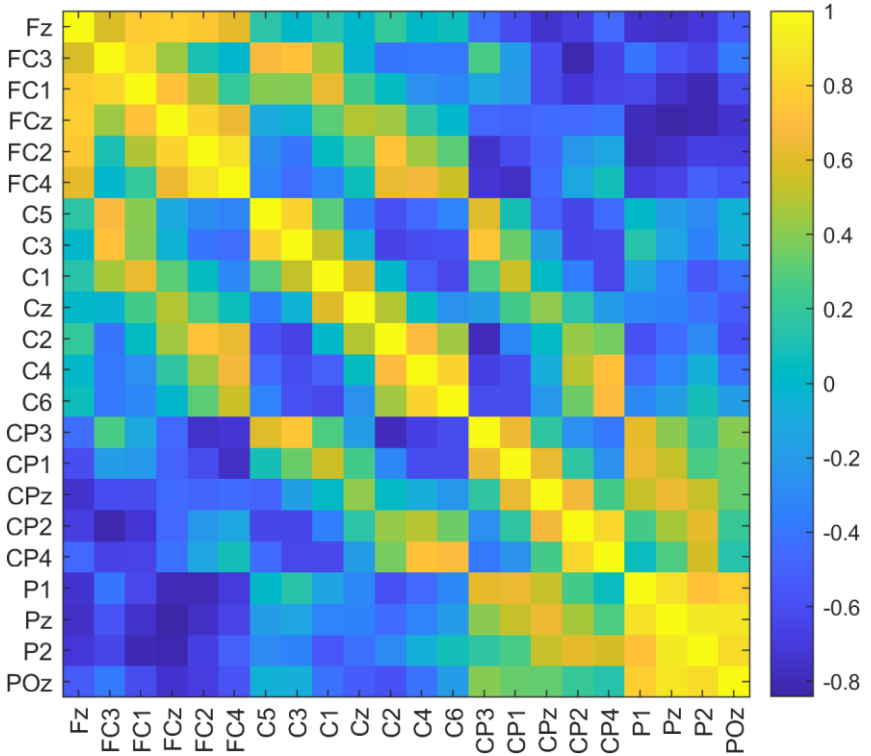
when $r > 0$ we say that the time series are **positively correlated**, and when $r < 0$ we say that they are **negatively correlated**.

Correlation

MATLAB provides the function corrcoef for the calculation of the Pearson's correlation coefficient.

To visualize a brain connectivity matrix:

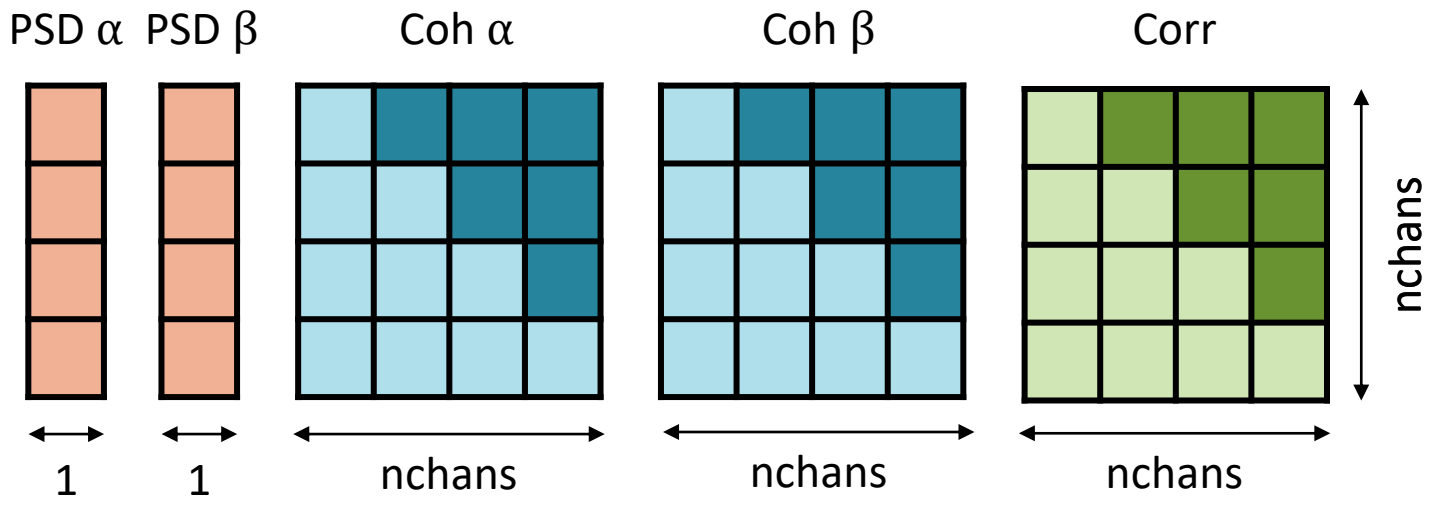
```
figure;
imagesc(coh_alfa_RH(:,:,1));
axis square;
colorbar;
xticks(1:1:EEG.nbchan);
yticks(1:1: EEG.nbchan);
xticklabels(chanlabels);
yticklabels(chanlabels);
xtickangle(90);
```



Feature storing

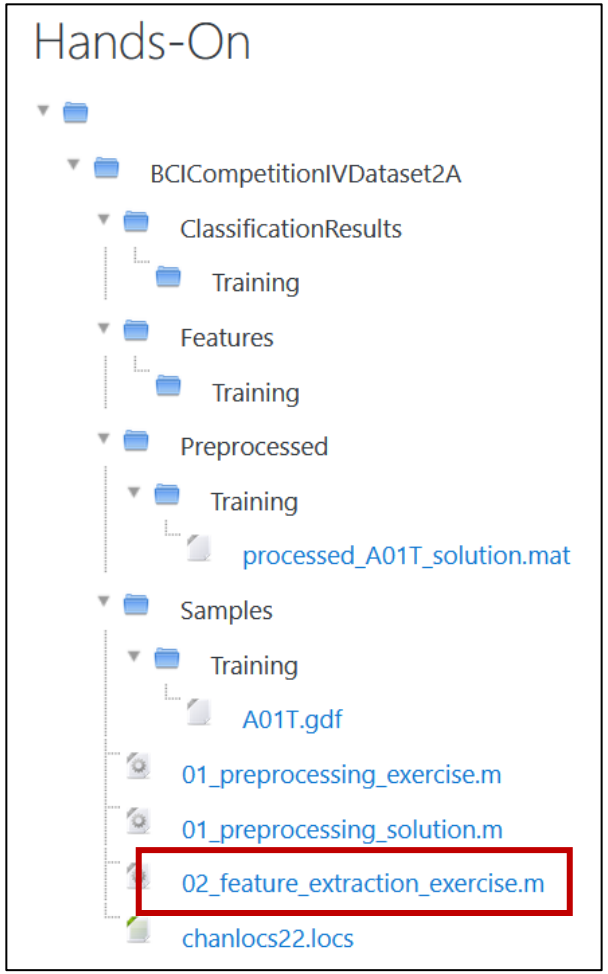
Summary of the seen features and their dimensionality.

For each event (“Right Hand” or “Both Feet”) and for each epoch (in this case, 72):




```
% Transform features in row
% vectors, one for each epoch.
% For PSD
PSD_RH_alfa =
psd_RH_alfa(:,:,1)';
% For connectivity
mask =
ones([EEG.nbchan,EEG.nbchan]);
mask = triu(mask,1);
COH_RH_alfa = coh_RH_alfa(:,:,1);
COH_RH_alfa =
COH_RH_alfa(logical(mask));
```

Exercise


Hands-On
BCICompetitionIVDataset2A
ClassificationResults
Training
Features
Training
Preprocessed
Training
processed_A01T_solution.mat
Samples
Training
A01T.gdf
01_preprocessing_exercise.m
01_preprocessing_solution.m
02_feature_extraction_exercise.m
chanlocs22.locs